VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
INSTITUTE OF COMPUTER SCIENCE
DEPARTMENT OF COMPUTATIONAL AND DATA MODELING

Project Report

Feature Engineering and Data Modeling for Spring4Shell (CVE-2022-22965) Detection using Random Forest Algorithm

Done by:
Evaldas Dmitrišin
Lukas Šerelis
Karolis Zabulis

Supervisor:
prof. PhD. Linas Bukauskas

Vilnius
2026

Contents

Abstract

This project aims to develop a machine learning-based detection system for the Spring4Shell vulnerability (CVE-2022-22965). We are using a containerized environment to simulate a vulnerable Spring Boot application and generated a dataset of normal and abnormal traffic. The most important part of the project is hybrid network flow feature engineering, where raw packet captures are transformed into structured data using both flow analysis and Layer 7 payload inspection. A Random Forest algorithm is then trained on these features to distinguish between normal HTTP interactions and exploit related patterns. The results demonstrate the effectiveness of combining flow-based metrics with textual keyword analysis for cyber attack detection.

Santrauka

Požymių inžinerija ir duomenų modeliavimas Spring4Shell (CVE-2022-22965) aptikimui naudojant „Random Forest“ algoritmą

Šio projekto tikslas yra sukurti mašininiu mokymusi pagrįstą aptikimo sistemą, skirtą Spring4Shell pažeidžiamumui (CVE-2022-22965). Naudojame konteinerizuotą aplinką, kad simuliuotume pažeidžiamą „Spring Boot“ programą ir sugeneravome įprasto ir neįprasto tinklo srauto duomenų rinkinį. Svarbiausia projekto dalis yra hibridinė tinklo srautų požymių inžinerija, kurios metu pirminiai paketų duomenys paverčiami į struktūrizuotus duomenis naudojant srautų analizės įrankius bei aplikacijų sluoksnio paketų turinio analizę. „Random Forest“ algoritmas apmokomas naudojant šiuos požymius, kad atskirtų įprastas HTTP sąveikas nuo su atakomis susijusių. Rezultatai parodo srautų metrikomis ir tekstinių raktažodžių analizės efektyvumą aptinkant kibernetines atakas.

Introduction

The Spring4Shell vulnerability (CVE-2022-22965) represents a significant threat to Java-based web applications, allowing for remote code execution by exploiting the way Spring handles data binding. While signature-based systems can detect known exploit strings, machine learning offers a more flexible approach by learning the underlying patterns of the attack. In this project we implement a full pipeline for vulnerability detection following the "Feature Engineering & Data Modeling" approach. The project is structured into three main phases:

- **Data Acquirement:** Setting up a vulnerable environment using Docker and simulating traffic patters using CRUD operations as harmless noise and remote code execution exploits as malicious samples.
- **Feature Engineering:** This phase transforms raw traffic into structured data by combining statistical flow metrics (durations, packet counts) with Layer 7 payload characteristics extracted from the HTTP requests.
- **Modeling:** Training a Random Forest algorithm to classify the extracted flows and evaluating its performance on VU MIF infrastructure.

All components, including the simulation, extraction, and training scripts, are designed to be fully executable on the VU MIF cloud infrastructure, providing a reproducible environment for evaluation.

1 Infrastructure and Traffic Simulation

To achieve reliable data modeling, a controlled environment was created on the VU MIF infrastructure using a Debian 13 virtual machine. All operations, from dependency installation to data collection, were automated using a suite of custom scripts.

1.1 Environment Setup

The target application is a Spring Boot web server vulnerable to CVE-2022-22965. It is containerized using `Docker` to ensure a reproducible and isolated environment. The setup process is managed by the following scripts:

- **`setup_victim.sh`:** This script automates the full environment preparation, including the installation of `Docker`, `tshark`, and `python3-requests`. It also configures `tshark` for non-root packet capture and launches the victim container at port 8080.
- **`setup_attacker.sh`:** Prepares the environment for the exploitation phase by ensuring the presence of `python3` and the `requests` library.
- **`setup_data.sh`:** A utility script that installs the necessary data science libraries (`nfstream`, `pandas`, `scikit-learn`) and prepares the system for feature extraction.

1.2 Traffic Generation

Data collection was structured into two distinct phases to create a balanced dataset for the machine learning model:

- **Normal traffic:** Legitimate user behavior is simulated using the `simulate_valid_traffic.sh` script. This script performs randomized CRUD operations for students, departments, and instructors. It also deliberately generates harmless "invalid" traffic (e.g., `/root/incorrect`) to teach the model that simple 404 errors are not necessarily attacks.
- **Abnormal Traffic:** The Spring4Shell exploit is executed using a `exploit.py` script and then attacker behavior is simulated using the `simulate_attacker_traffic.py` script. This script simulates various post-exploitation activities, including OS reconnaissance, database credential theft, and full database extraction.

1.3 Feature Extraction

The network traffic was captured using the `capture_traffic.sh` script, which provides an interactive interface to select the network interface and defines the number of packets and files to be recorded. The transformation of raw PCAP data into a structured dataset is handled by `extract_features.py`. This script serves as the bridge between `NFStream` and `tshark`:

- It utilizes `NFStreamer` to extract statistical flow metrics.
- It calls `tshark` as a subprocess to extract TCP hex payloads, which are then decoded into ASCII.
- It enforces a 2000-character limit per file to maintain performance while ensuring the Random Forest algorithm has access to the most relevant L7 keyword signatures.

2 Feature Engineering and Dataset Preparation

The most important challenge of detecting Spring4Shell using machine learning is transforming raw network packets into a format that a Random Forest algorithm can interpret. This part of the project, called feature engineering, involves converting the network traffic into a table of numbers.

2.1 Flow Extraction Process

We developed a custom extraction pipeline that synchronizes flow statistics with raw packet content. We used `NFStream` to group individual packets into bidirectional sessions and calculate statistical metrics. Simultaneously, we used `tshark` to extract the raw TCP hex payloads from the same `.pcapng` files.

Because Spring4Shell exploits are often spread across multiple packets, our script decodes the hex data into ASCII and aggregates the first 2000 characters of the capture file's payload. This aggregated text is then mapped back to the flows generated by `NFStream`, creating a rich dataset that combines traditional flow metrics with actual application layer data.

2.2 Key Features for Detection

Initially, the model relied on flow-based metrics. However, to improve robustness against Spring4Shell, we expanded the feature set to include Layer 7 payload characteristics:

- **Payload Content:** Using `TfidfVectorizer`, we analyze the raw HTTP payload for specific keywords associated with the Spring4Shell exploit, such as `classLoader`, `cmd`, `runtime`, and `jsp`.
- **Feature Ratios:** We introduced calculated ratios to normalize the data:
  1. *Packet Ratio:* Ratio of source-to-destination packets.
  2. *ACK Ratio:* Frequency of acknowledgment packets.
  3. *Duration Ratio:* To evaluate the timing of the interaction.
- **Protocol Signatures:** We monitor the frequency of specific characters like `"="`, which often appears in the malicious POST requests used to inject the exploit.

3 Implementation and Training

Our implementation was done in two main parts: setting up the environment to get the data and then using Python to build the detection model.

3.1 Tools and Environment

To implement the full pipeline from raw traffic to a functioning classifier, we utilized several specialized tools:

- **Docker:** Used to containerize the vulnerable Spring Boot application. This allowed for an isolated environment that could be instantly reset between attack simulations.

- **Tshark:** A critical tool used for two purposes: capturing live traffic on the virtual interface and performing deep packet inspection to extract raw hex payloads from `.pcapng` files.
- **NFStream:** A high-performance Python framework used to process packet captures into flow-based statistical data. It provided the foundation for our numerical features.
- **Scikit-learn:** Rather than a simple model, we used the `Pipeline` library to integrate text processing (`TfidfVectorizer`) and numerical classification into a single workflow.
- **Python Libraries** `Pandas` was used for data manipulation and CSV generation, while the `Requests` library was the core engine for both our legitimate traffic and attacker simulation scripts.
- **MIF Infrastructure:** All development and training were performed on Debian 13 virtual machines within the VU MIF cloud, ensuring environment compatibility.

3.2 Training the Model

The detection system is implemented as a Scikit-learn `Pipeline` utilizing a `ColumnTransformer`. This architecture allows the model to handle two different types of data simultaneously:

- **Numerical Data:** Flow metrics and the three calculated ratios (Packet, ACK, and Duration ratios).
- **Textual Data:** Raw payload strings processed through `TfidfVectorizer` to identify exploit-related keywords and protocol signatures.

After converting our packet capture files into CSV file, we had a dataset containing both normal and malicious examples. To ensure the model could generalize across different network conditions, we simulated traffic using varying inter-packet delays. The dataset was constructed from a total of 100 packet capture files, each containing 5000 packets. The dataset was labeled (*0 for normal, 1 for malicious*) and split into a training set (60%) to teach the model, a validation set (20%) used to fine-tune the model parameters, and a testing set (20%) to evaluate its performance.

By using the Random Forest algorithm, we did not have to manually write rules to detect the attack. Instead, the model learned by itself that certain packet sizes and session lengths usually mean an exploit is happening.

4 Results

The model achieves an accuracy of approximately 84%. A key finding during evaluation was that the model initially relied heavily on *Flow Duration*. However, since timing can vary significantly due to network jitter, we prioritized the inclusion of payload keywords (`cmd`, `class`, etc.).

The introduction of `TfidfVectorizer` features proved crucial. Even when packet sizes were similar between normal and malicious traffic, the presence of specific L7 keywords allowed the Random Forest to maintain a high detection rate. Current testing shows that while the payload extraction adds some processing overhead, it significantly reduces false positives compared to a purely flow-based approach.